Roles & Access Structure

AI Career Growth & Talent Intelligence Platform — Backend
Reference for the frontend team: who can do what, and how to build around it

---

1. The 5 roles, at a glance

Every user has exactly one role, stored as a plain string on the account and included as a claim in the JWT. There is no multi-role/permission-group system — role fully determines what a user can see and do.

Role	Who they are	Can self-register?
candidate	A job seeker building a profile and applying/matching against roles.	Yes — default role
recruiter	Company staff who source and evaluate candidates.	No — admin-created only
hr_reviewer	Company staff who screen/review candidates.	No — admin-created only
employer	Company staff representing a hiring organization.	No — admin-created only
administrator	Platform operator. Manages users and has all staff permissions.	No — email allowlist only

Important for UI design: recruiter, hr_reviewer, and employer currently have **identical permissions** — the backend does not differentiate access between them. They exist as distinct labels for organizational/reporting reasons, not for different capabilities. Design your UI around two permission tiers today — "candidate" and "company staff" (recruiter/hr_reviewer/employer) — plus administrator, rather than building three separate staff experiences. It's fine to show the specific role label (e.g. in a profile badge) since it does come back from the API — just don't gate any feature on which of the three it is.

2. Authentication and finding out the current user's role

2.1 Login — form-encoded, not JSON:

```
POST /api/v1/auth/login
Content-Type: application/x-www-form-urlencoded

username=<email>&password=<password>
```

Response:

```
{
"access_token": "<jwt>",
"refresh_token": "<jwt>",
"token_type": "bearer"
}
```

2.2 Getting the role. The access token's payload does contain a `role` claim, but treat that as a convenience, not a source of truth — the frontend should not trust an unverified client-side JWT decode for anything permission-critical. Immediately after login (and on app load, if a token is already stored), call:

```
GET /api/v1/auth/me
Authorization: Bearer <access_token>
```

Response:

```
{
"id": "uuid",
"email": "...",
"full_name": "...",
"role": "candidate | recruiter | hr_reviewer | employer | administrator",
"is_active": true,
"is_verified": true
}
```

Use this response as the canonical source for role-based routing/rendering decisions. Store it in whatever global auth/user state your app uses (context, store, etc.) alongside the tokens.

Token refresh: access tokens expire in 30 minutes by default. Use `POST /auth/refresh` with the refresh token to get a new pair before/when the access token expires; the old refresh token is revoked on use, so always store the new one. A revoked or expired refresh token returns 401 — at that point, redirect to login.

Deactivated accounts: if an admin deactivates a user (`is_active: false`), their next login attempt gets a 403 with detail "Account is deactivated" — surface this as a distinct message from "wrong password", not a generic login failure.

3. Permission matrix

"Own" means the endpoint only ever operates on the logged-in user's own data (there is no id param to target someone else). "Any" means the endpoint takes a target id and works across all candidates/users.

Capability	candidate	company staff*	administrator
View/edit own profile, skills, education, experience	Yes (own)	—	—
Upload/manage own resumes	Yes (own)	—	—
Compute/view own benchmark matches	Yes (own)	—	—
Read benchmarks	Yes	Yes	Yes
Create/edit/deactivate benchmarks	No	Yes	Yes
Search/browse all candidate profiles	No	Yes (any)	Yes (any)
Compute/view any candidate's matches	No	Yes (any)	Yes (any)
Create recruiter/hr_reviewer/employer/candidate/admin accounts	No	No	Yes
List all users / filter by role	No	No	Yes
Reassign any user's role	No	No	Yes
Activate/deactivate any user	No	No	Yes

`*company staff = recruiter, hr_reviewer, employer (identical permissions today).`

4. Suggested page/navigation structure per role

This is a suggested information architecture based on what each role can actually do — not a mandated design. Adjust to your product's UX conventions; the point is which capabilities exist to build pages around.

4.1 Candidate

- **Dashboard** — profile completeness, quick links to resume/skills/matches.
- **My Profile** — edit personal info, career preferences; manage skills, education, experience lists.
- **My Resumes** — upload, view versions, see parsing status, download, delete.
- **Career Matches** — trigger/view benchmark matches: match score, readiness score, gap summary (missing skills/certs, experience gap).
- **Browse Benchmarks** (optional, read-only) — see what benchmarks exist/what roles require, since candidates can read (not manage) benchmarks.

4.2 Company staff (recruiter / hr_reviewer / employer)

- **Candidate Directory** — paginated, filterable search (industry, functional area, experience level, experience-years range, skill). Each row shows name + email (now returned directly by the search endpoint) plus profile summary fields.
- **Candidate Detail** — full profile view (read-only from this role's perspective — staff cannot edit a candidate's own data), plus a "compute/view matches" action against benchmarks.
- **Benchmarks** — full CRUD: list, create, edit, deactivate. This is a shared management surface across all three staff labels.

- There is currently no distinct "my queue" or role-specific workflow in the backend for recruiter vs. hr_reviewer vs. employer — build one shared staff experience rather than three.

4.3 Administrator

- Everything company staff can see, plus a dedicated **Admin > Users** section:
- **User list** — filterable by role, paginated (limit/offset).
- **Create user** — form for email/password/full name/role (any of the 5 roles) — this is how recruiter/hr_reviewer/employer/administrator accounts actually get created; there's no public signup for them.
- **Reassign role** — change any existing user's role (e.g. correct a batch of staff created with the default role, or promote/demote a user). Disable this action on the admin's own row in the UI — the API rejects it with 400 anyway, but it's better UX to just not offer it.
- **Activate / deactivate** — toggle a user's access without deleting the account.

5. API endpoints available to each role

All paths are prefixed with `/api/v1`.

5.1 Public (no auth) — used by the login/register screens

Method	Path	Notes
POST	`/auth/register`	Candidate self-signup. role defaults to candidate; any other role value is rejected (422) unless the email happens to be on the server's admin allowlist.
POST	`/auth/login`	form-urlencoded. Returns access_token + refresh_token.
POST	`/auth/refresh`	Body {refresh_token}. Returns a new token pair.
POST	`/auth/logout`	Body {refresh_token}. Revokes it — call on explicit user logout.
POST	`/auth/password-reset/request`	Body {email}. Always 202 (no account-enumeration signal in the response).
POST	`/auth/password-reset/confirm`	Body {token, new_password}. Token delivery isn't wired up yet (see Section 6).

5.2 Any authenticated user

Method	Path	Notes
GET	`/auth/me`	Current user's id/email/full_name/role/is_active/is_verified — call this to drive role-based UI.
GET	`/benchmarks`	List/browse benchmarks (query: category, level, functional_area, is_active).
GET	`/benchmarks/{id}`	Single benchmark detail.

5.3 candidate only

Method	Path	Notes
GET / PUT	`/candidates/me`	Own profile.
POST / DELETE	`/candidates/me/skills[/{id}]`	Own skills list.
POST / DELETE	`/candidates/me/education[/{id}]`	Own education list.
POST / DELETE	`/candidates/me/experience[/{id}]`	Own experience list.
POST	`/resumes/upload`	multipart/form-data, field 'file'. pdf/doc/docx, 10MB max.
GET	`/resumes`	List own resume versions.
GET	`/resumes/{id}`	Parsing status + parsed_data for one resume.
GET	`/resumes/{id}/download`	Streams the raw file.
POST	`/resumes/{id}/reparse`	Re-run parsing.
DELETE	`/resumes/{id}`	Delete a resume version.
POST	`/matching/me/compute`	Optional ?benchmark_id=... Computes match(es) for the logged-in candidate.
GET	`/matching/me`	List the candidate's own computed matches.

5.4 recruiter, hr_reviewer, employer (identical set)

Method	Path	Notes
GET	`/candidates`	Paginated, filterable candidate directory: {items, total, limit, offset}. Each item includes full_name/email.
GET	`/candidates/{id}`	Single candidate's full profile (read-only), enriched with full_name/email.
POST / PUT / DELETE	`/benchmarks[/{id}]`	Create/edit/deactivate benchmarks.
POST	`/matching/candidates/{id}/compute`	Compute match(es) for a specific candidate.
GET	`/matching/candidates/{id}`	View a specific candidate's computed matches.

5.5 administrator only

Method	Path	Notes
POST	`/admin/users`	Create a user of any role. role defaults to recruiter if omitted.
GET	`/admin/users`	List all users, optional ?role= filter, paginated.
PATCH	`/admin/users/{id}/role`	Body {role}. Reassign a user's role. 400 if targeting yourself.
PATCH	`/admin/users/{id}/deactivate`	Disable a user's access. 400 if targeting yourself.
PATCH	`/admin/users/{id}/activate`	Re-enable a deactivated user.

6. Edge cases and states to design for

- **403 vs 401.** A 401 means the token is missing/invalid/expired — redirect to login. A 403 means the token is valid but the role doesn't permit the action — show an "access denied" state, don't treat it as a session problem (don't log the user out).
- **Role changes mid-session take effect immediately — not on next refresh.** Every request re-loads the user from the database and authorizes off that live value; the JWT's role claim is not what's checked, so an unrefreshed access token is not enough to retain (or be denied) access after an admin changes a role. In practice: the moment an admin reassigns a user's role, that user's very next request with their existing, unrefreshed token already reflects the new permissions — a candidate promoted to recruiter can call staff endpoints immediately, and immediately loses access to candidate-only ones. Design for this directly rather than building a "please refresh" affordance: if your UI caches the role from the initial /auth/me call, a request can now legitimately 403 (or newly succeed) without any token change on your end — handle that the same way you'd handle any other permission response, not as a stale-session bug.
- **Empty candidate directory results.** GET /candidates returns {items: [], total: 0, ...} for no matches — distinguish this from a loading or error state in the UI.
- **Resume parsing is async.** After upload, parsing_status starts as "pending", moves to "processing", then "completed" or "failed" — poll GET /resumes/{id} (every 1-2s is reasonable) and show a progress/spinner state until it resolves. parsing_method will show "rules" (the LLM fallback path is currently disabled) — this is expected, not an error state to surface to the user.
- **Password reset has no email delivery yet.** POST /auth/password-reset/request always returns 202 regardless of whether the email exists (this is intentional, to avoid leaking which emails are registered) — but there is currently no email actually sent with the reset token. Don't build a "check your email" flow that promises delivery until this is wired up server-side; flag this to product/backend before shipping a password-reset UI to real users.
- **New accounts are auto-verified.** is_verified exists on the user model but nothing currently blocks login for an unverified account, and there is no email-confirmation step in this backend — don't build a "verify your email" gate expecting the backend to enforce it.
- **Company-staff self-registration will always fail (422).** Don't build a public signup form offering recruiter/hr_reviewer/employer/administrator as a role choice — there's no such flow. The only paths to those roles are the admin-allowlist (administrator) and the admin Create User screen (Section 4.3).

Generated for the frontend team. If anything here is out of date with the running API, the OpenAPI schema at /docs is the source of truth — flag the discrepancy so this guide can be updated.